PRÁCTICAS PROFESIONALIZANTES II

Testing y seguridad en aplicaciones ReactJS

Material complementario de estudio

Calidad del software · Pruebas unitarias y de componentes · Seguridad web

Explicaciones progresivas · Código comentado · Ejercicios aplicados al catálogo de productos · Autoevaluación

Diseño y Desarrollo Web

Edición de estudio · Octubre de 2026

Cómo utilizar este material

Propósito, recorrido y conocimientos previos

Este documento amplía el material complementario de ReactJS. Primero presenta el concepto de testing y las principales estrategias de prueba; luego profundiza en pruebas unitarias de funciones y pruebas de componentes con React. La segunda parte introduce riesgos frecuentes de seguridad en aplicaciones frontend y prácticas para reducirlos.

Los ejemplos mantienen el catálogo de productos utilizado durante el curso: **TarjetaProducto.jsx**, el formulario de búsqueda y el hook **useProductos.js**. La intención es probar comportamientos concretos de una aplicación conocida, no memorizar herramientas aisladas.

Objetivos de aprendizaje

• Explicar por qué probar una aplicación reduce riesgos, sin afirmar que elimina todos los defectos.

• Distinguir pruebas unitarias, de integración, de sistema/end-to-end, de regresión y exploratorias.

• Preparar Vitest, JSDOM y React Testing Library en un proyecto Vite.

• Escribir pruebas para funciones JavaScript y para el comportamiento visible de componentes React.

• Comprender el rol de MSW al simular respuestas HTTP sin modificar el código de producción.

• Reconocer riesgos como XSS, URLs peligrosas, dependencias vulnerables y confianza indebida en el cliente.

Recorrido

Sección	Contenido
1	Qué es testing y tipos de pruebas
2	Pruebas unitarias: estructura, casos y funciones
3	Testing de componentes React con Vitest y RTL
4	Pruebas asíncronas y simulación de API con MSW
5	Seguridad web en el frontend
6	Práctica integradora, autoevaluación y glosario

> Forma de estudio sugerida: antes de ejecutar cada test, escribir qué comportamiento se espera. Después, ejecutar la prueba, observar el resultado y cambiar deliberadamente una condición para comprobar que el test detecta el cambio.

Conocimientos previos: funciones, módulos, arreglos, objetos, promesas, HTML semántico, React con props y estado, eventos, fetch y el entorno Vite presentado en el material anterior.

1. ¿Qué es el testing de software?

Verificar comportamientos y reducir incertidumbre

El **testing de software** es el proceso de evaluar una aplicación para encontrar defectos y obtener evidencia sobre su comportamiento. Una prueba compara un resultado observado con un resultado esperado para un escenario definido.

En una aplicación React, por ejemplo, podemos comprobar que una tarjeta muestre el nombre de un producto, que el precio se formatee correctamente o que el botón para seleccionar un producto esté deshabilitado cuando no hay stock.

¿Por qué probar?

• **Detectar errores temprano:** un test pequeño suele ayudar a localizar la causa con más precisión que un fallo observado al final del desarrollo.

• **Proteger cambios:** después de refactorizar una función o un componente, la suite permite comprobar que los comportamientos importantes siguen funcionando.

• **Documentar expectativas:** los casos de prueba expresan qué debe ocurrir con entradas válidas, vacías, límite o inválidas.

• **Facilitar el mantenimiento:** las pruebas confiables dan señales rápidas cuando una modificación rompe un contrato existente.

Testing no es lo mismo que depurar

Actividad	Propósito	Ejemplo
Testing	Revelar si un escenario cumple lo esperado.	Probar que precio negativo sea rechazado.
Depuración (debugging)	Investigar y corregir la causa de un defecto.	Encontrar por qué una función acepta ese precio.
Revisión estática	Examinar código sin ejecutarlo.	ESLint detecta una regla o patrón riesgoso.

Un test no demuestra que no existan errores

Una prueba cubre los escenarios que fueron diseñados. Si no se probaron entradas inesperadas, errores de red o permisos, estos defectos pueden seguir presentes. La calidad se construye combinando pruebas, revisión de código, accesibilidad, observabilidad y controles de seguridad.

> Pregunta guía: ¿qué podría hacer una persona usuaria, una API o un dato inesperado que cambie el resultado de esta función o pantalla?

2. Tipos de testing

Diferentes alcances para diferentes preguntas

Tipo	Qué comprueba	Ejemplo en el catálogo
Unitario	Una unidad pequeña de lógica, aislada de dependencias externas.	formatearPrecio(1500) devuelve una cadena con moneda.
De componentes	El comportamiento visible de un componente en un DOM de prueba.	TarjetaProducto deshabilita el botón sin stock.
Integración	La colaboración entre módulos o capas.	App y useProductos presentan el resultado de una petición simulada.
Sistema / end-to-end (E2E)	Un flujo completo de la aplicación en un entorno cercano al real.	Abrir catálogo, buscar un producto y seleccionarlo en el navegador.
Regresión	Que una función que antes funcionaba siga funcionando después de cambios.	Un cambio de formato no rompe los precios ya soportados.
Exploratorio / manual	Investigar el comportamiento sin limitarse a casos predefinidos.	Probar combinaciones de búsqueda, teclado y estados de carga.
Aceptación	Que el producto cumpla criterios acordados con quienes lo solicitan.	El catálogo permite encontrar y seleccionar productos según la consigna.

Unitario, integración y E2E: una comparación

La diferencia principal es el **alcance**, no la importancia. Las pruebas unitarias suelen ser rápidas y precisas; las de integración comprueban contratos entre piezas; las E2E verifican flujos completos, pero suelen requerir más tiempo y configuración.

Nivel	Velocidad típica	Aislamiento	Fallos que puede detectar
Unitario	Alta	Alto	Reglas de cálculo y casos límite.
Integración	Media	Parcial	Incompatibilidades entre módulos o API.
E2E	Menor	Bajo	Problemas de navegación o flujo completo.

También existen clasificaciones por objetivo, como pruebas funcionales y no funcionales. Las funcionales comprueban qué hace el sistema; las no funcionales pueden evaluar rendimiento, accesibilidad, compatibilidad, usabilidad o seguridad.

> Para esta clase, el foco será testing unitario y de componentes. La simulación de red con MSW se presenta como extensión para comprobar la integración de la interfaz con una API.

3. Anatomía de una prueba

Preparar, ejecutar y comprobar

Una forma común de organizar una prueba es el patrón **Arrange – Act – Assert** (Preparar – Actuar – Verificar). No es una regla sintáctica obligatoria, sino una estructura que ayuda a leer qué se está probando.

```
import { describe, it, expect } from "vitest";
import { sumar } from "./sumar.js";

describe("sumar", () => {
  it("suma dos números", () => {
    // Arrange: preparar las entradas
    const a = 2;
    const b = 3;

    // Act: ejecutar la unidad
    const resultado = sumar(a, b);

    // Assert: verificar el resultado esperado
    expect(resultado).toBe(5);
  });
});
```

Conceptos que aparecen en el ejemplo

Concepto	Responsabilidad
describe	Agrupa pruebas relacionadas bajo un nombre.
it / test	Define un escenario de prueba.
expect	Recibe el valor observado para formular una comprobación.
toBe	Compara el resultado con un valor esperado mediante igualdad estricta.
Caso de prueba	Una situación concreta: entrada, acción y resultado esperado.
Suite	Conjunto de pruebas ejecutadas para un módulo o proyecto.

Qué casos conviene incluir

- **Camino feliz:** la entrada común y válida.
- **Límites:** cero, cadena vacía, primer/último elemento, valores mínimos y máximos.
- **Entradas inválidas:** tipo incorrecto, valor ausente o fuera del rango permitido.
- **Errores externos:** respuesta HTTP fallida, tiempo de espera o datos inesperados, cuando aplique.

> Un buen test tiene un propósito identificable. Si falla, el mensaje debería ayudar a comprender qué comportamiento dejó de cumplirse.

4. Preparar Vitest en un proyecto Vite

Herramientas de la práctica

Vitest ejecuta pruebas y proporciona aserciones. Se integra con la configuración de Vite. **JSDOM** implementa una versión de las APIs del navegador en memoria. **React Testing Library (RTL)** permite renderizar componentes y consultar la interfaz. **user-event** simula interacciones de usuario.

Herramienta	Función principal
Vitest	Ejecutar pruebas, agrupar casos y verificar resultados.
JSDOM	Ofrecer un DOM de prueba sin abrir un navegador gráfico.
@testing-library/react	Renderizar componentes React y consultar el resultado.
@testing-library/user-event	Simular clics, escritura, foco y otras interacciones.
@testing-library/jest-dom	Agregar aserciones legibles para estados del DOM.
MSW	Interceptar peticiones HTTP en la capa de red para probar estados de API.

Instalación

```
npm install -D vitest jsdom @testing-library/react
npm install -D @testing-library/dom @testing-library/user-event
npm install -D @testing-library/jest-dom
```

Los paquetes son dependencias de desarrollo. Se instalan en el proyecto existente, dentro de la carpeta que contiene package.json.

Configurar Vitest

Si el proyecto ya tiene vite.config.js, incorporar la configuración de test a la existente en lugar de crear dos configuraciones que compitan.

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    setupFiles: "./src/test/setup.js",
    globals: false,
    clearMocks: true
  }
});
```

Archivo src/test/setup.js

```
import "@testing-library/jest-dom/vitest";
```

Agregar un script a package.json

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "vitest",
    "test:run": "vitest run"
  }
}
```

> Usar npm test para ejecutar Vitest en modo interactivo. Para una ejecución que finalice —por ejemplo, en una revisión puntual— usar npm run test:run. Los scripts existentes de package.json deben conservarse.

5. Pruebas unitarias de funciones

Probar lógica independiente de la interfaz

Las funciones puras son un buen punto de partida: reciben entradas y devuelven resultados sin modificar datos externos ni depender de la pantalla. Se pueden probar sin montar React ni crear un DOM.

Función del catálogo: formatear un precio

```
// src/utils/precio.js
export function formatearPrecio(valor) {
  return new Intl.NumberFormat("es-AR", {
    style: "currency",
    currency: "ARS"
  }).format(valor);
}
```

La función del ejemplo base convierte un valor numérico en una cadena monetaria. El test debe comprobar el contrato observable; como el formato exacto puede variar según el entorno de internacionalización, puede ser más robusto verificar que la salida contiene el valor esperado y el símbolo/indicador monetario.

```
// src/utils/precio.test.js
import { describe, it, expect } from "vitest";
import { formatearPrecio } from "./precio.js";

describe("formatearPrecio", () => {
  it("incluye el valor monetario", () => {
    const resultado = formatearPrecio(1500);

    expect(resultado).toContain("1.500");
    expect(resultado).toMatch(/\$/);
  });

  it("devuelve texto", () => {
    expect(typeof formatearPrecio(0)).toBe("string");
  });
});
```

Función de negocio: filtrar productos

```
// src/utils/productos.js
export function filtrarProductos(productos, consulta) {
  const termino = consulta.trim().toLowerCase();

  return productos.filter(producto =>
    producto.nombre.toLowerCase().includes(termino)
  );
}
```

```
// src/utils/productos.test.js
import { describe, it, expect } from "vitest";
import { filtrarProductos } from "./productos.js";

const productos = [
  { id: "1", nombre: "Tomate", precio: 1500 },
  { id: "2", nombre: "Lechuga", precio: 900 }
];

describe("filtrarProductos", () => {
  it("encuentra coincidencias sin distinguir mayúsculas", () => {
    expect(filtrarProductos(productos, "TOM"))
      .toEqual([{ id: "1", nombre: "Tomate", precio: 1500 }]);
  });

  it("ignora espacios al inicio y al final", () => {
    expect(filtrarProductos(productos, "  lechuga "))
      .toHaveLength(1);
  });

  it("devuelve todos los productos con una consulta vacía", () => {
    expect(filtrarProductos(productos, ""))
      .toHaveLength(2);
  });

  it("devuelve una lista vacía si no hay coincidencias", () => {
    expect(filtrarProductos(productos, "banana"))
      .toEqual([]);
  });
});
```

Qué conviene observar

• toEqual compara estructuras como objetos y arreglos.

• toHaveLength verifica la cantidad de elementos.

• Cada test nombra una regla concreta: mayúsculas, espacios, consulta vacía o ausencia de coincidencias.

> Si una función es responsable de validar datos, documentar también qué debe ocurrir con null, undefined o tipos inesperados. No agregar esos casos sin decidir antes cuál es el contrato esperado.

6. Pruebas de componentes React

Verificar la experiencia observable

Una prueba de componente renderiza el componente en un DOM de prueba y comprueba qué puede percibir una persona usuaria. En lugar de inspeccionar variables internas o detalles del árbol de React, se consultan roles, nombres accesibles, etiquetas y texto visible.

Componente de referencia: TarjetaProducto

```
// src/components/TarjetaProducto.jsx
export default function TarjetaProducto({
  producto,
  onSeleccionar
}) {
  const disponible = producto.stock > 0;

  return (
    <article>
      <h2>{producto.nombre}</h2>
      <p>{disponible ? "Disponible" : "Sin stock"}</p>
      <button
        type="button"
        disabled={!disponible}
        onClick={() => onSeleccionar(producto.id)}
      >
        Seleccionar {producto.nombre}
      </button>
    </article>
  );
}
```

Este componente recibe un producto y una función callback. No necesita conocer cómo se guarda la selección: su responsabilidad es presentar los datos y comunicar la acción.

Prueba del botón cuando no hay stock

```
// src/components/TarjetaProducto.test.jsx
import { describe, it, expect, vi } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import TarjetaProducto from "./TarjetaProducto.jsx";

describe("TarjetaProducto", () => {
  it("deshabilita la selección si no hay stock", () => {
    const producto = {
      id: "2",
      nombre: "Lechuga",
      precio: 900,
      stock: 0
    };

    render(
      <TarjetaProducto
        producto={producto}
        onSeleccionar={() => {}}
      />
    );

    expect(
      screen.getByRole("button", {
        name: "Seleccionar Lechuga"
      })
    ).toBeDisabled();
  });

  it("notifica la selección si hay stock", async () => {
    const user = userEvent.setup();
    const onSeleccionar = vi.fn();
    const producto = {
      id: "1",
      nombre: "Tomate",
      precio: 1500,
      stock: 8
    };

    render(
      <TarjetaProducto
        producto={producto}
        onSeleccionar={onSeleccionar}
      />
    );

    await user.click(
      screen.getByRole("button", {
        name: "Seleccionar Tomate"
      })
    );

    expect(onSeleccionar).toHaveBeenCalledWith("1");
  });
});
```

Lectura del test

Parte	Qué hace
render(...)	Monta el componente y sus elementos en el DOM de prueba.
screen.getByRole(...)	Busca un botón mediante su rol y nombre accesible.
toBeDisabled()	Comprueba un estado observable del control.
vi.fn()	Crea una función simulada para observar si se llamó y con qué argumentos.
user.click(...)	Simula una interacción de usuario y espera sus pasos asíncronos.

> El test no pregunta si la variable disponible vale true o si el componente usa una expresión ternaria. Pregunta si la persona puede seleccionar un producto sin stock: ese es el comportamiento que importa.

7. Consultas accesibles en React Testing Library

Elegir la consulta según lo que se necesita comprobar

Consulta	Si no encuentra el elemento	Uso habitual
getByRole / getByLabelText	Lanza un error.	El elemento debe existir ya.
queryByRole / queryByText	Devuelve null.	Comprobar que un elemento no está presente.
findByRole / findByText	Devuelve una promesa y reintenta hasta el límite.	El elemento aparece de forma asíncrona.

Jerarquía recomendada

Priorizar consultas que se parezcan a cómo se identifica el contenido en la interfaz:

• **getByRole:** botones, encabezados, enlaces, alertas y otros roles accesibles.

• **getByLabelText:** campos asociados a una etiqueta de formulario.

• **getByText:** contenido textual visible cuando no hay una consulta semántica mejor.

• **data-testid:** reservar para casos en los que no exista una alternativa semántica adecuada.

Ejemplo con formulario

```
render(<FormularioProducto onCrear={() => {}} />);

const campo = screen.getByRole("textbox", {
  name: "Nombre del producto"
});
await user.type(campo, "Tomate");

await user.click(
  screen.getByRole("button", { name: "Agregar" })
);
```

Para que la consulta por nombre funcione, el input debe estar asociado a una etiqueta mediante htmlFor e id, o disponer de otro nombre accesible válido. Esto promueve pruebas más robustas y, al mismo tiempo, mejores interfaces.

Evitar pruebas atadas a la implementación

• No buscar clases CSS para comprobar el comportamiento funcional.

• No depender de nombres de variables de estado ni de la estructura interna del componente.

• No comprobar detalles irrelevantes del HTML si el requisito es de comportamiento.

• Sí verificar textos de error, estado disabled, resultado visible y respuesta a acciones.

> La accesibilidad y el testing se refuerzan mutuamente: una interfaz con roles y etiquetas adecuados suele ser más fácil de probar y de utilizar con tecnologías de asistencia.

8. Probar formularios y comportamiento asíncrono

Acciones realistas y estados visibles

Para campos controlados, conviene escribir con user-event y comprobar el resultado visible. No basta con invocar directamente un manejador: la prueba debería acercarse al recorrido que realiza una persona.

```
it("muestra un error si el nombre está vacío", async () => {
  const user = userEvent.setup();

  render(<FormularioProducto onCrear={() => {}} />);

  await user.click(
    screen.getByRole("button", { name: "Agregar" })
  );

  expect(
    screen.getByRole("alert")
  ).toHaveTextContent("Ingresar el nombre del producto.");
});
```

En código real, este ejemplo supone que el formulario presenta el mensaje de error con role="alert", como en el material de ReactJS. Las pruebas deben reflejar el contrato real del componente.

Cargas, errores y resultados vacíos

Cuando el componente consulta una API, hay varios estados distintos que deben considerarse:

Estado	Qué verificar
Cargando	Se presenta un indicador como role="status".
Éxito con datos	Los productos recibidos aparecen en la interfaz.
Éxito sin datos	Se comunica que el catálogo está vacío o que no hay coincidencias.
Error HTTP o de red	Se muestra un mensaje comprensible con role="alert".

Esperar elementos que aparecen después

```
expect(
  await screen.findByRole("heading", { name: "Tomate" })
).toBeInTheDocument();
```

findBy... espera durante un tiempo acotado a que el elemento aparezca. En cambio, getBy... falla de inmediato si el elemento aún no existe. No se recomienda agregar esperas arbitrarias con setTimeout: es preferible esperar la condición observable.

> Las pruebas asíncronas deben esperar el resultado que importa, no un número fijo de milisegundos. Esto reduce la fragilidad ante diferencias de rendimiento.

9. Simular una API con Mock Service Worker

Probar la interfaz sin depender del servidor real

Mock Service Worker (MSW) intercepta solicitudes en la capa de red y permite definir respuestas de prueba. De este modo, el código de la aplicación sigue utilizando fetch y el hook useProductos sin reemplazar la lógica de producción.

Instalar MSW

```
npm install -D msw
```

Definir un handler

```
// src/test/handlers.js
import { http, HttpResponse } from "msw";

export const handlers = [
  http.get("http://localhost:3000/productos", () => {
    return HttpResponse.json([
      { id: "1", nombre: "Tomate", precio: 1500, stock: 8 }
    ]);
  })
];
```

Crear el servidor de prueba

```
// src/test/server.js
import { setupServer } from "msw/node";
import { handlers } from "./handlers.js";

export const server = setupServer(...handlers);
```

Conectar el servidor a Vitest

```
// src/test/setup.js
import "@testing-library/jest-dom/vitest";
import { afterAll, afterEach, beforeAll } from "vitest";
import { server } from "./server.js";

beforeAll(() => server.listen({ onUnhandledRequest: "error" }));
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

Si setup.js ya importa jest-dom, agregar allí el resto del código. La opción onUnhandledRequest: "error" ayuda a detectar solicitudes que no fueron contempladas en la prueba.

Simular un error HTTP 500

```
import { http, HttpResponse } from "msw";

server.use(
  http.get("http://localhost:3000/productos", () => {
    return new HttpResponse(null, { status: 500 });
  })
);
```

Después de renderizar el componente que carga productos, el test debe esperar el mensaje de error que presenta App.jsx, por ejemplo con screen.findByRole("alert"). El texto exacto depende de cómo la aplicación traduzca el estado de error.

> MSW no reemplaza la necesidad de probar el backend. En este contexto permite comprobar que la interfaz interpreta correctamente respuestas exitosas, vacías y fallidas.

10. Introducción a la seguridad web frontend

La interfaz forma parte de la superficie de ataque

La seguridad web busca proteger la confidencialidad, integridad y disponibilidad de una aplicación, además de los datos de las personas usuarias. El frontend participa en esa protección, pero no puede ser la única barrera: el navegador se ejecuta en un entorno que controla el usuario.

Las validaciones visuales mejoran la experiencia, pero un usuario puede modificar el DOM, cambiar el estado de la aplicación o enviar solicitudes HTTP manualmente. Por eso, el backend debe validar los datos y autorizar cada operación sensible de forma independiente.

Principio de no confianza en el cliente

> Una regla aplicada solo en React no protege un recurso del servidor. Ocultar un botón o impedir un envío desde el formulario no reemplaza la autorización del backend.

Control en frontend	Límite	Control complementario
Validar un formulario	Puede omitirse con una petición manual.	Validación y reglas de negocio en el servidor.
Ocultar acciones según el rol	El usuario puede alterar la interfaz.	Autorización en cada endpoint.
Guardar datos en estado React	El estado no es una fuente de verdad persistente.	Verificar permisos y consistencia en backend.
Variables de entorno de Vite	Las variables incluidas en el bundle son visibles.	Secretos solo en infraestructura de servidor.

Riesgos que se estudiarán

• **XSS:** contenido controlado por un atacante termina interpretándose como código en el navegador.

• **URLs peligrosas:** enlaces que usan esquemas no permitidos, como javascript:.

• **Dependencias comprometidas o vulnerables:** paquetes de terceros incorporan riesgos en la cadena de suministro.

• **Exposición de credenciales:** tokens o secretos almacenados o publicados en lugares accesibles al cliente.

• **Clickjacking y transporte inseguro:** la configuración del sitio puede permitir incrustación no deseada o comunicaciones sin protección adecuada.

11. XSS, JSX y contenido no confiable

Escapar por defecto no significa que cualquier uso sea seguro

Cross-Site Scripting (XSS) ocurre cuando datos controlados por un atacante se interpretan como contenido ejecutable en el navegador. Puede permitir acciones en nombre de una persona usuaria, alterar contenido o intentar acceder a información disponible para los scripts de la página.

La interpolación de texto en JSX

```
const comentario = "<img src=x onerror=alert('prueba') />";

return <p>{comentario}</p>;
```

En este caso React trata la variable como texto y escapa los caracteres relevantes: el navegador no interpreta la cadena como un elemento HTML. Esta protección es una ventaja importante del uso normal de JSX.

Una zona de riesgo: HTML explícito

```
// Evitar si el contenido no es confiable:
<div dangerouslySetInnerHTML={{ __html: contenidoExterno }} />
```

dangerouslySetInnerHTML indica que se desea interpretar marcado HTML. Si la aplicación necesita esa funcionalidad, el contenido debe sanitizarse con una biblioteca especializada como DOMPurify antes de insertarlo. La sanitización reduce el riesgo, pero debe aplicarse correctamente y mantenerse actualizada.

No inyectar contenido mediante el DOM

Evitar asignar datos externos directamente a innerHTML o insertar nodos con contenido no validado a través de referencias. Estas vías pueden eludir las protecciones habituales de React.

Validar protocolos de URLs

```
export function esUrlSegura(valor) {
  try {
    const url = new URL(valor);
    return url.protocol === "https:" || url.protocol === "http:";
  } catch {
    return false;
  }
}
```

La función acepta solo URLs absolutas con protocolos HTTP o HTTPS. Si el producto necesita enlaces relativos, debe definirse y probarse explícitamente esa regla. No conviene asumir que una URL es segura por tener apariencia de texto común.

Pruebas sugeridas para una función de URL

- Aceptar una URL HTTPS válida.
- Aceptar una URL HTTP si el requisito la permite.
- Rechazar un esquema javascript:.
- Rechazar una cadena que no pueda analizarse como URL.

> No confiar en el contenido por el solo hecho de que venga de una API, un CMS o un campo de texto. Los datos externos deben tratarse según el contexto en el que se van a utilizar.

12. Dependencias, secretos y sesiones

Reducir riesgos fuera del código de los componentes

Cadena de suministro de NPM

Los proyectos React dependen de paquetes de terceros. Un paquete puede tener vulnerabilidades conocidas, estar abandonado, ser reemplazado por uno malicioso o confundirse con otro de nombre similar. El riesgo puede aparecer durante la instalación, la compilación o la ejecución.

Práctica	Para qué sirve
Revisar el nombre y origen	Reducir errores de typosquatting y dependencias innecesarias.
Conservar package-lock.json	Registrar versiones resueltas para instalaciones reproducibles.
npm ci	Instalar en entornos automatizados usando el lockfile y sin modificarlo.
npm audit	Consultar vulnerabilidades conocidas en dependencias.
npm doctor	Revisar aspectos del entorno local de npm, como caché y configuración.
Actualizar con criterio	Evaluar correcciones, cambios incompatibles y resultados de pruebas.

```
npm audit
npm doctor
npm ci
```

Estos comandos no garantizan que un paquete sea seguro ni reemplazan la revisión de dependencias. Evaluar los resultados antes de aplicar cambios automáticos.

Nunca publicar secretos en el frontend

Las variables que Vite incorpora al bundle del cliente pueden inspeccionarse desde los archivos compilados o las herramientas del navegador. Por eso, ninguna variable de entorno destinada al frontend debe contener contraseñas, claves privadas o tokens que deban mantenerse secretos.

```
// Ejemplo conceptual: dato público de configuración
const apiBaseUrl = import.meta.env.VITE_API_BASE_URL;

// No colocar secretos privados en variables VITE_*
```

Tokens y almacenamiento del navegador

Cualquier script que se ejecute en el mismo origen puede leer datos guardados en localStorage o sessionStorage. Si una aplicación almacena allí credenciales de sesión, una vulnerabilidad XSS puede aumentar su impacto. Para aplicaciones con autenticación, conviene evaluar una estrategia de sesión del lado del servidor con cookies configuradas correctamente, por ejemplo HttpOnly, Secure y SameSite, según la arquitectura y las necesidades del producto.

> No existe una solución universal de autenticación aplicable a todas las arquitecturas. La decisión debe considerar backend, dominio, CORS, CSRF, duración de sesión, renovación y modelo de amenazas.

13. Automatizar controles de seguridad

Linters, cabeceras y configuración de producción

Análisis estático

ESLint y sus plugins pueden detectar patrones problemáticos antes de ejecutar la aplicación. Las reglas disponibles dependen de la configuración y no reemplazan el criterio de revisión. Es útil revisar advertencias relacionadas con HTML no sanitizado, atributos peligrosos y dependencias.

Cabeceras HTTP

Cabecera	Objetivo general
Content-Security-Policy (CSP)	Limitar los orígenes y tipos de recursos que puede cargar o ejecutar la página.
Strict-Transport-Security (HSTS)	Indicar al navegador que utilice HTTPS para el sitio durante el período configurado.
X-Frame-Options	Restringir la incrustación en frames para mitigar ciertos escenarios de clickjacking.
X-Content-Type-Options: nosniff	Evitar ciertos tipos de interpretación MIME no solicitada por el servidor.

Estas cabeceras se configuran normalmente en el servidor, proxy o plataforma que entrega la aplicación. Una CSP requiere diseño y pruebas cuidadosas; una política mal configurada puede bloquear recursos legítimos. Para nuevas configuraciones, revisar la compatibilidad actual de los navegadores y las directivas recomendadas.

Lista breve de revisión antes de entregar

- Las entradas se validan en frontend para orientar al usuario y en backend para proteger los datos.
- No se interpreta HTML no confiable sin sanitización.
- Las URLs externas se validan según una lista de protocolos permitidos.
- No se incluyen claves privadas ni secretos en el bundle de Vite.
- Se revisan las dependencias y se conserva el archivo de bloqueo.
- Las operaciones sensibles verifican autenticación y autorización en el servidor.
- Se revisan HTTPS y las cabeceras de seguridad en el entorno de producción.

14. Práctica integradora

Aplicar testing y seguridad al catálogo

Trabajar sobre el proyecto React del curso. No es necesario reescribir el catálogo: se agregan pruebas y se revisan puntos concretos de seguridad.

Actividad A — pruebas unitarias

• Crear la función filtrarProductos(productos, consulta) en src/utils/productos.js.

• Escribir pruebas para coincidencia parcial, mayúsculas/minúsculas, espacios, consulta vacía y ausencia de coincidencias.

• Ejecutar npm run test:run y corregir los fallos.

Actividad B — prueba de componente

• Crear TarjetaProducto.test.jsx con React Testing Library.

• Comprobar que el botón esté deshabilitado cuando stock sea 0.

• Comprobar que al hacer clic en un producto disponible se llame onSeleccionar con su id.

• Usar consultas accesibles por rol y nombre; no seleccionar por clase CSS.

Actividad C — estados de API

• Agregar MSW para simular la respuesta de GET /productos.

• Probar la carga exitosa con productos y la respuesta exitosa con arreglo vacío.

• Simular HTTP 500 y verificar que la interfaz muestre un mensaje de error accesible.

Actividad D — revisión de seguridad

• Buscar en el proyecto usos de dangerouslySetInnerHTML e innerHTML.

• Revisar cómo se construyen enlaces a partir de datos dinámicos y definir protocolos aceptados.

• Ejecutar npm audit y leer el resultado sin aplicar cambios automáticos a ciegas.

• Comprobar que ningún secreto se encuentre en variables VITE_* ni en archivos subidos al repositorio.

• Explicar por qué ocultar un botón no reemplaza la autorización del backend.

> Criterio de logro: además de conseguir que las pruebas pasen, cada estudiante debe explicar qué comportamiento protege cada test y qué riesgo de seguridad reduce cada control.

15. Autoevaluación

Responder antes de consultar el repaso

1. ¿Qué diferencia hay entre testing y depuración?

2. ¿En qué se distinguen una prueba unitaria, una de integración y una E2E?

3. ¿Qué significan Arrange, Act y Assert?

4. ¿Por qué conviene probar una función pura sin montar React?

5. ¿Qué comprueba el test de TarjetaProducto cuando stock es 0?

6. ¿Cuándo utilizar getByRole, queryByRole y findByRole?

7. ¿Por qué user-event puede ser más representativo que llamar directamente a un manejador?

8. ¿Qué aporta MSW al probar un hook que usa fetch?

9. ¿Qué protección ofrece React al renderizar texto mediante llaves en JSX?

10. ¿Por qué dangerouslySetInnerHTML requiere un tratamiento especial?

11. ¿Por qué una validación de formulario no constituye una barrera de autorización?

12. ¿Qué información nunca debe publicarse dentro de variables VITE_*?

13. ¿Qué diferencia hay entre npm audit y npm doctor?

14. ¿Qué objetivo general tienen CSP y HSTS?

Repaso de respuestas

1. Testing evalúa escenarios frente a resultados esperados; depuración investiga y corrige la causa de un defecto.

2. Unitario prueba una unidad pequeña; integración comprueba la colaboración entre piezas; E2E recorre un flujo completo.

3. Preparar datos, ejecutar la acción y verificar el resultado.

4. Aísla la lógica y permite comprobar entradas y casos límite con rapidez.

5. Que la acción de selección no esté disponible cuando no hay stock.

6. getBy busca presencia inmediata y falla si no existe; queryBy permite comprobar ausencia; findBy espera una aparición asíncrona.

7. Simula interacciones más cercanas a las de una persona, incluyendo eventos de foco y teclado.

8. Intercepta peticiones en la capa de red y permite controlar respuestas exitosas, vacías y fallidas sin modificar el código de producción.

9. React escapa el contenido interpolado como texto, evitando que una cadena se interprete normalmente como HTML.

10. Porque permite insertar HTML que el navegador interpreta; contenido no confiable debe sanitizarse antes.

11. Puede eludirse desde el navegador o mediante peticiones directas; el servidor debe validar y autorizar.

12. Claves privadas, contraseñas, credenciales y tokens secretos.

13. npm audit consulta vulnerabilidades conocidas en dependencias; npm doctor revisa el entorno y la configuración local de npm.

14. CSP limita fuentes de recursos y ejecución; HSTS indica que el sitio debe utilizar HTTPS.

16. Glosario y referencias

Términos esenciales y documentación para profundizar

Término	Definición breve
Aserción	Comprobación que compara un resultado observado con una expectativa.
Callback	Función entregada a otra pieza de código para que la invoque.
DOM	Representación estructurada del documento que el navegador expone a JavaScript.
Mock	Reemplazo controlado o función simulada utilizada para observar o controlar una interacción.
MSW	Herramienta para interceptar solicitudes de red y simular respuestas.
Regresión	Fallo en un comportamiento que antes funcionaba.
Sanitización	Proceso de eliminar o neutralizar contenido no permitido para un contexto.
Suite	Conjunto de pruebas relacionadas.
XSS	Vulnerabilidad por la que contenido no confiable se ejecuta como script en el navegador.

Referencias oficiales y lecturas

[1] Vitest — Getting Started: https://vitest.dev/guide/

[2] React Testing Library — Introduction: https://testing-library.com/docs/react-testing-library/intro/

[3] Testing Library — Queries: https://testing-library.com/docs/queries/about/

[4] Testing Library — user-event: https://testing-library.com/docs/user-event/intro/

[5] jest-dom — Custom matchers: https://github.com/testing-library/jest-dom

[6] Mock Service Worker — Documentation: https://mswjs.io/docs/

[7] React — Writing Markup with JSX: https://react.dev/learn/writing-markup-with-jsx

[8] OWASP — Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[9] OWASP — HTML5 Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

[10] npm — npm audit: https://docs.npmjs.com/cli/commands/npm-audit

[11] npm — npm ci: https://docs.npmjs.com/cli/commands/npm-ci

[12] MDN — Content Security Policy: https://developer.mozilla.org/docs/Web/HTTP/CSP

[13] MDN — Strict-Transport-Security: https://developer.mozilla.org/docs/Web/HTTP/Headers/Strict-Transport-Security

> Alcance: este material es introductorio y didáctico. Las herramientas, versiones y configuraciones pueden evolucionar. Para proyectos reales, consultar la documentación oficial vigente y revisar la arquitectura completa, especialmente los controles que deben ejecutarse en el backend.